



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The **State Space** is the set of all distinct physical configurations and legal transitions that exist in the environment. In this grid world, it is finite and bounded by at most $\text{width} \times \text{height}$ discrete coordinates.

The **Search Tree** is an abstract representation of candidate paths generated by the search algorithm starting from the initial state root. Nodes in the search tree represent paths through the state space. While the physical state space is finite, the search tree can be infinite because loops, cycles, or repeated visits create duplicate paths to the exact same state coordinate.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

In graph search, the `reached` (visited) set prevents the search algorithm from re-expanding the same physical state along redundant or cyclic paths.

Without a `reached` set, DFS operates as a pure tree search. Because grid actions are bidirectional (e.g., executing `Up` then `Down` returns to the initial cell), DFS gets trapped in an **infinite loop** alternating between adjacent nodes along a single infinite branch, or it quickly crashes from a recursion/memory call stack overflow without ever finding the goal.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS Optimality: BFS expands nodes layer by layer using a FIFO queue, visiting states in increasing order of path depth. Because every step in an unweighted grid carries an identical step cost of 1, path depth equals total path cost ($g(n)$). Therefore, the first time BFS finds the goal, it is mathematically guaranteed to be the shortest, optimal path.

DFS Suboptimality: DFS explores greedily down the deepest path first using a LIFO stack. It commits to the first available direction at each branch until hitting a dead end before backtracking. As a result, it frequently takes long, winding paths around the perimeter of the grid rather than walking directly toward a visible target adjacent to it.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS Memory Bottleneck:

BFS has an exponential space complexity of $O(b^d)$, where b is the branching factor (up to 4 in a grid) and d is the depth of the shallowest goal. BFS must keep all generated frontier nodes and exploration boundaries in RAM simultaneously. On a large 1000 $\times$ 1000 grid where the goal depth d can be hundreds of steps away, 4^d causes an immediate Out-Of-Memory (OOM) system crash.

DFS Space Efficiency:

DFS has a linear space complexity of $O(b \cdot m)$, where m is the maximum path depth. DFS only keeps track of the current active branch and unexplored sibling alternatives along that branch. Even if $m = 10^6$ on a 1000 $\times$ 1000 grid, linear space consumption remains small and fits comfortably in RAM without running out of memory.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING